Five Whys you should start your next SQL Data Source using "new blank query"

- Unlock SQL that can be accessed by humans, AI and RPA in Power Platform.
- Productize initial report building steps to accelerate MVP creation.
- Covers gaps in Fabric Notebook/pySpark capabilities when Fabric capacity is not available.
- Without ai coding agents or Copilot in Power BI, data lineage and model documentation becomes mostly automated.
- Leverage full capabilities of Query Folding and push complex transforms upstream into SQL vs in Power Query

ƒxQuerySQLite Python m function

Use Native Database Connector instead of Python when possible

```
(sqlQuery as text) =>
let
    PythonScript =
        "import sqlite3#(lf)" &
        "import pandas as pd#(lf)#(lf)" &
        "conn = sqlite3.connect('" & oakhavendatabasepath & "')#(lf)" &
        "sql = '''" & sqlQuery & "'''#(lf)" &
        "df = pd.read_sql_query(sql, conn)#(lf)" &
        "conn.close()",

    Source = Python.Execute(PythonScript),
    DataTable = Source{[Name="df"]}[Value]
in
    DataTable
```

Fact Sales m Code Example

```
let
    Source = fxQuerySQLite(query_fact_sales),
    #"Changed Type" = Table.TransformColumnTypes(Source,{{"datekey", Int64.Type}, {"order_id", Int64.Type},
{"order_line_id", Int64.Type}, {"customer_id", Int64.Type}, {"product_id", Int64.Type}, {"employee_id", Int64.Type},
{"quantity", type number}, {"unit_price", type number}, {"discount_pct", type number}, {"net_amount", type number}})
in #"Changed Type"
```

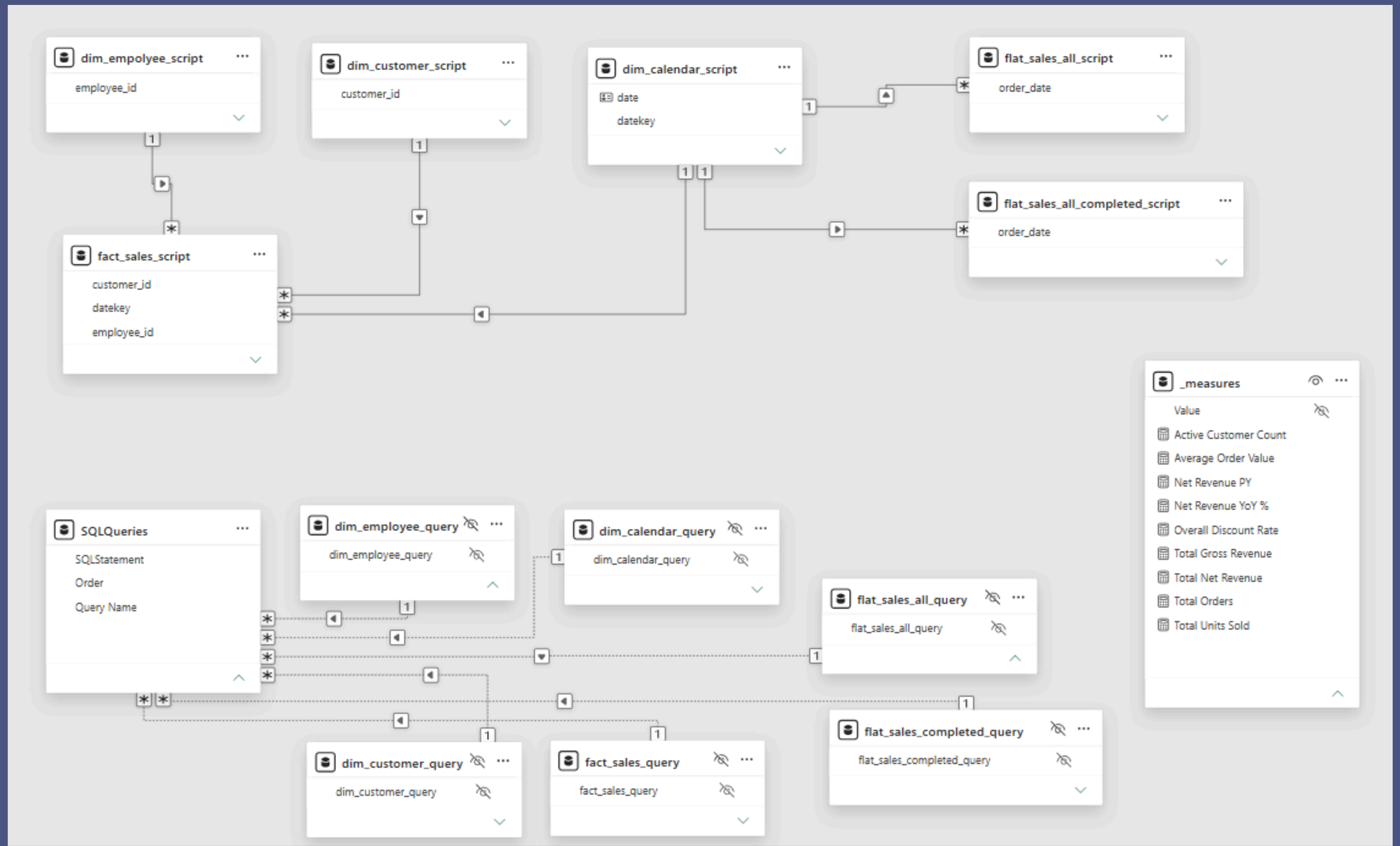
Native Database Query Folding Example

Only as performant as the query. Folding won't fix a slow query

```
let
    // 1. Connect to the database server
    Source = Sql.Database("localhost", "AdventureWorks"),

    // 2. Execute custom SQL while keeping Query Folding alive
    CustomSQL = Value.NativeQuery(
        Source,
        "SELECT CustomerID, FirstName, LastName, Country, TotalPurchases FROM Sales.vCustomers",
        null,                  // 3rd arg: null (no bind parameters)
        [EnableFolding = true]  // 4th arg: keeps Query Folding active!
    ),

    // 3. Downstream steps WILL fold into the database as a subquery
    #"Filtered to USA" = Table.SelectRows(CustomSQL, each [Country] = "USA"),
    #"Top Customers"   = Table.Sort(#"Filtered to USA", {{"TotalPurchases", Order.Descending}})
in
    #"Top Customers"
```


dim_empolyee_script
employee_id
dim_customer_script
customer_id
dim_calendar_script
date
datekey
flat_sales_all_script
order_date
flat_sales_all_completed_script
order_date
fact_sales_script
customer_id
datekey
employee_id
_measures
Value
Active Customer Count
Average Order Value
Net Revenue PY
Net Revenue YoY %
Overall Discount Rate
Total Gross Revenue
Total Net Revenue
Total Orders
Total Units Sold
SQLQueries
SQLStatement
Order
Query Name
dim_employee_query
dim_employee_query
dim_calendar_query
dim_calendar_query
flat_sales_all_query
flat_sales_all_query
dim_customer_query
dim_customer_query
fact_sales_query
fact_sales_query
flat_sales_completed_query
flat_sales_completed_query

Query Name	SQLStatement
dim_calendar	SELECT DISTINCT d.datekey, d.date, d.year, d.month, d.month_name, d.quarter, d.day_name, d.is_weekend FROM dim_date d ORDER BY d.datekey ;
dim_customer	SELECT DISTINCT c.customer_id, c.full_name, c.email, c.state, c.customer_segment FROM dim_customer c WHERE c.customer_id IN (SELECT customer_id FROM fact_sales ORDER BY order_date DESC, order_id, order_line_id) ;
dim_employee	SELECT DISTINCT e.employee_id, e.full_name, e.department, e.region FROM dim_employee e WHERE e.employee_id IN (SELECT employee_id FROM fact_sales ORDER BY order_date DESC, order_id, order_line_id) ;

Query Name	SQLStatement
fact_sales	SELECT f.order_id, f.order_line_id, f.customer_id, f.product_id, f.employee_id, f.datekey, f.quantity, f.unit_price, f.discount_pct, f.net_amount, f.payment_method, f.order_status, f.channel FROM fact_sales f ORDER BY f.order_date DESC, f.order_id, f.order_line_id .

Query Name	SQLStatement
flat_sales_all	SELECT f.order_id, f.order_line_id, f.order_date, c.full_name AS customer_name, p.product_name, p.category AS product_category, f.quantity, f.net_amount, f.channel FROM fact_sales f LEFT JOIN dim_customer c ON f.customer_id = c.customer_id LEFT JOIN dim_product p ON f.product_id = p.product_id WHERE f.order_status = 'Completed' ORDER BY f.order_date DESC .
flat_sales_completed	SELECT f.order_id, f.order_line_id, f.order_date, c.full_name AS customer_name, p.product_name, p.category AS product_category, f.quantity, f.net_amount, f.channel FROM fact_sales f LEFT JOIN dim_customer c ON f.customer_id = c.customer_id LEFT JOIN dim_product p ON f.product_id = p.product_id WHERE f.order_status = 'Completed' ORDER BY f.order_date DESC .